

CaSSius now supports full headless PDF creation from JATS XML

Cite as: Martin Paul Eve, "CaSSius now supports full headless PDF creation from JATS XML", *eve.gd: Martin Paul Eve*, June 19, 2016, <https://doi.org/10.59348/6rkea-5q639>.

 CaSSius now supports full headless PDF creation from JATS XML

CaSSius is the PDF typesetter that I am building as part of my work for the Andrew W. Mellon Foundation grant to Birkbeck for the Open Library of Humanities. CaSSius allows for true XML-first workflows.

#Background CaSSius is called CaSSius (with that capitalization) because it uses a feature of “CSS” called Regions. Regions are an experimental and unsupported technology that allow the specification of “regions” (unsurprisingly) between which you can flow text. So, imagine you had an unspecified quantity of text. What you want to do is to create enough A4 pages that this content can be flowed between. CSS Regions theoretically allows us to do this. We can specify A4 regions (pages) and tell the browser to flow text between them.

The way that CaSSius works is as follows:

1. We have a JATS XML import procedure (XSLT) that takes the XML and produces an HTML document that is marked up in a way that our javascript can understand.
2. The javascript calls François Remy’s polyfill that adds in support for regions to any WebKit browser (more on this below)
3. Our javascript then waits (not very patiently) for the polyfill to do its job. Once that’s done, our javascript calculates whether we need more or fewer pages and adds or subtracts them as necessary.

This works fine in a browser and has done for some time. It creates nicely printable documents. But, what we couldn't do, was just have a neat tool that we can run from the command line that will produce the PDF. I didn't know why or what was causing this, only that when run in Chrome or Firefox, all was fine, but the second we were on the command line, a 25 page document would take upwards of 10 minutes to build.

#The fix

Until today, I had about 90% of this project in a good state. As above, what I couldn't get to work, though, was any kind of command-line tool to print a PDF. Every single implementation would crash. I've spent days on end thinking about how to fix this and hit a dead end every time. Except today, when I refused to be defeated and started to dig into the polyfill code.

It turns out that the problem was that the polyfill was exponentially passing the `*` selector to various match functions as the document grew, thereby consuming system resources and eventually dying hard, with no vengeance.

A simple check to ensure that neither `*` nor `*`, were added to any match tests did the trick:

```
if(selector != "*" && selector.indexOf("*",") < 0) {
```

#So now we can print from the command line The output of:

```
./wkhtmltopdf --javascript-delay 15000 --no-stop-slow-scripts -L 0 -R 0 -B 0 -T 0  
http://localhost:8000/ ~/result.pdf
```

[produces this PDF](#). Tada, JATS to PDF.